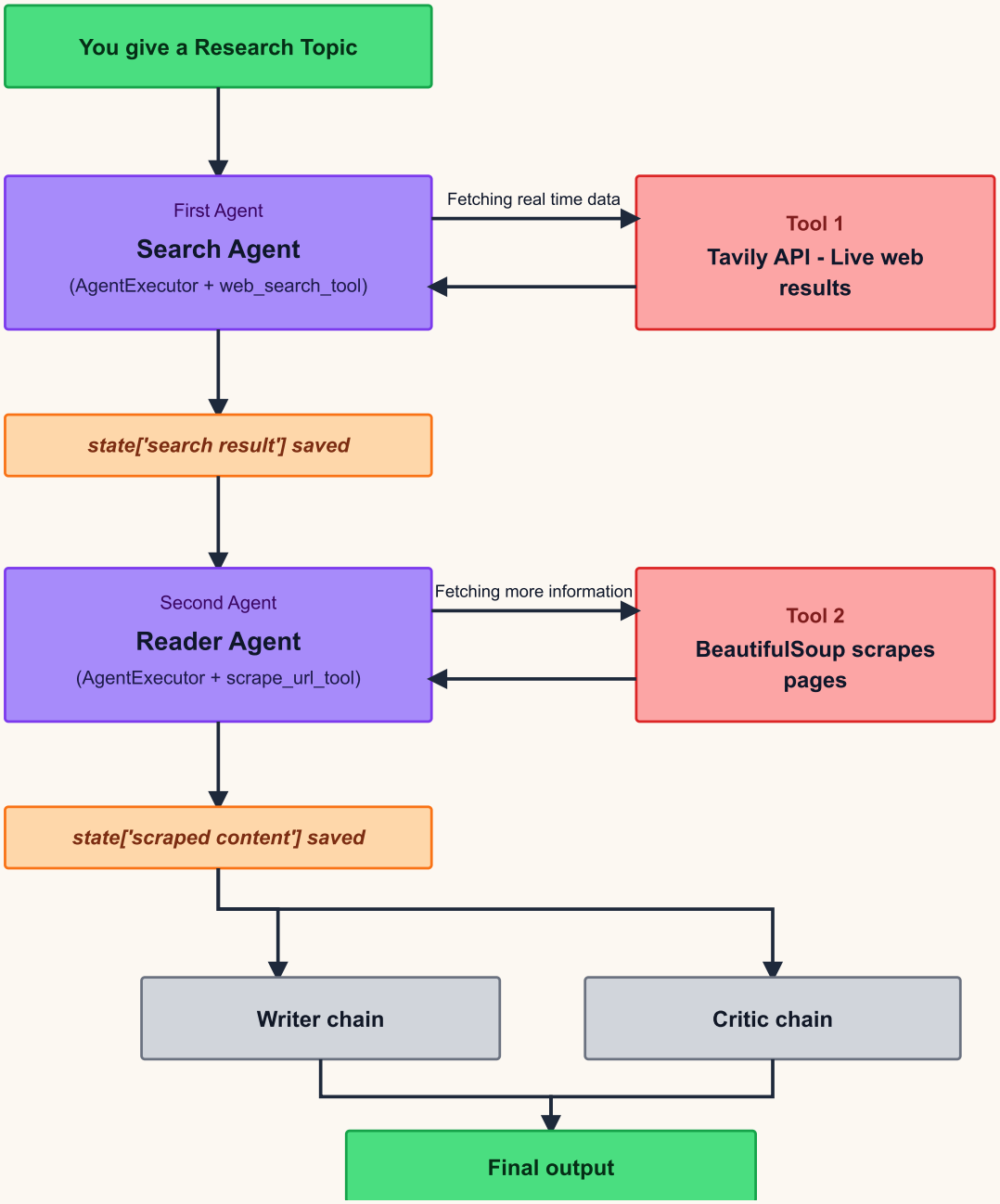
Multi-Agent Research Architecture

System Diagram & Execution Step Breakdown

Pipeline System Flowchart



Implementation Walkthrough

Detailed step-by-step development process extracted from documentation

Execution Steps

Step 1 — Environment Setup

Create a virtual environment, install all the libraries from `requirements.txt` and create the `.env` file with your OpenAI and Tavily API keys.

Step 2 — Create tools.py

Define reusable tools using the `@tool` decorator. First the `web_search` tool which talks to the Tavily API and fetches live search results from the internet. Second the `scrape_url` tool which takes a URL, visits that page and extracts clean readable text from it using BeautifulSoup.

Step 3 — Create agents.py

This is the heart of the project. We will build 4 things here. First the **Search Agent** using `create_agent()` from `langchain.agents` — just pass the model and tools and it returns a ready-to-use agent graph powered by LangGraph internally (no AgentExecutor or hub prompts needed). Second the **Reader Agent** using the same `create_agent()` pattern but with the `scrape_url` tool. Third the **Writer Chain** using the modern LCEL pipe syntax — `prompt | llm | StrOutputParser()` — which takes all the research and writes a full report. Fourth the **Critic Chain** again using LCEL pipe which reads the report and gives a score and feedback.

Step 4 — Create pipeline.py

This is the supervisor. We will write one function called `run_research_pipeline` that calls all 4 agents and chains in the correct order and passes results between them using a shared state dictionary. The agents use message-based input/output — we send `{"messages": [("user", "...")]}` and read the response from `result["messages"][-1].content`. At the end of each step it will print the output in the terminal so students can see exactly what each agent is doing.

Step 5 — Run and Test

Run `python pipeline.py` in the terminal, enter a research topic and watch all 4 agents work one by one — search, read, write, review — and print the final report with critic feedback right in the terminal.